Unit 1

INTRODUCTION TO OOP AND JAVA FUNDAMENTALS **10**

Object Oriented Programming - Abstraction – objects and classes - Encapsulation- Inheritance - Polymorphism- OOP in Java – Characteristics of Java – The Java Environment - Java Source File -Structure – Compilation. Fundamental Programming Structures in Java – Defining classes in Java – constructors, methods -access specifiers - static members –Comments-- JavaDoc comments , Data Types, Variables, Operators, Control Flow, Arrays , Packages

What is Object Oriented programming ?

Object-oriented programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data and code: data in the form of fields and code, in the form of procedures .

- Object Oriented Programming solves the real world problems in simple way.
- OOP helps to manage complexity.
- Object-oriented programming organizes a program around its data (that is, objects)

Objects and Classes

- Real world Entity is an Object
- Object - A thing that you want to store and process data about.

- Structure to store it’s properties – Class
- Template for creating objects and code written by the programmer is called class.

Real Wold Example of Classes And Objects

Class - Cars

Class - Machine

Real Life Problem

Program to Store Data of All the Persons Present in India

Complex??? Yes but for Other Languages Not for OOP

Class : Person

Properties

- Name
- Age
- Gender

Functions / Actions

- Eating
- Walking
- Sleeping

Real World Entities of Class - Object

Name: Ben
Age: 21
Gender: Male

Name: Emma
Age: 18
Gender: Female

Class Person

Code written by a programmer

Properties

FirstName:
LastName:
Gender:
DateOfBirth:
Occupation:

Operations

PayWages()
BookHoliday()
Appraisal()
Promotion()
SaveDetails()

Creating an object from a class is called instantiation

Assigning values to each object

```
P5 = New Person("Kevin", "Drumm", "Male", #11.02.1962#, "Teacher")
```

Abstraction

- Abstraction is the process of hiding the internal details of an application from the outer world. Abstraction is used to describe things in simple terms.

- To simplify reality.

Real Life Problem

Class SBI

addUser()

interestRate() = 8 %

Class HDFC

addUser()

interestRate() = 9 %

Class ICICI

addUser()

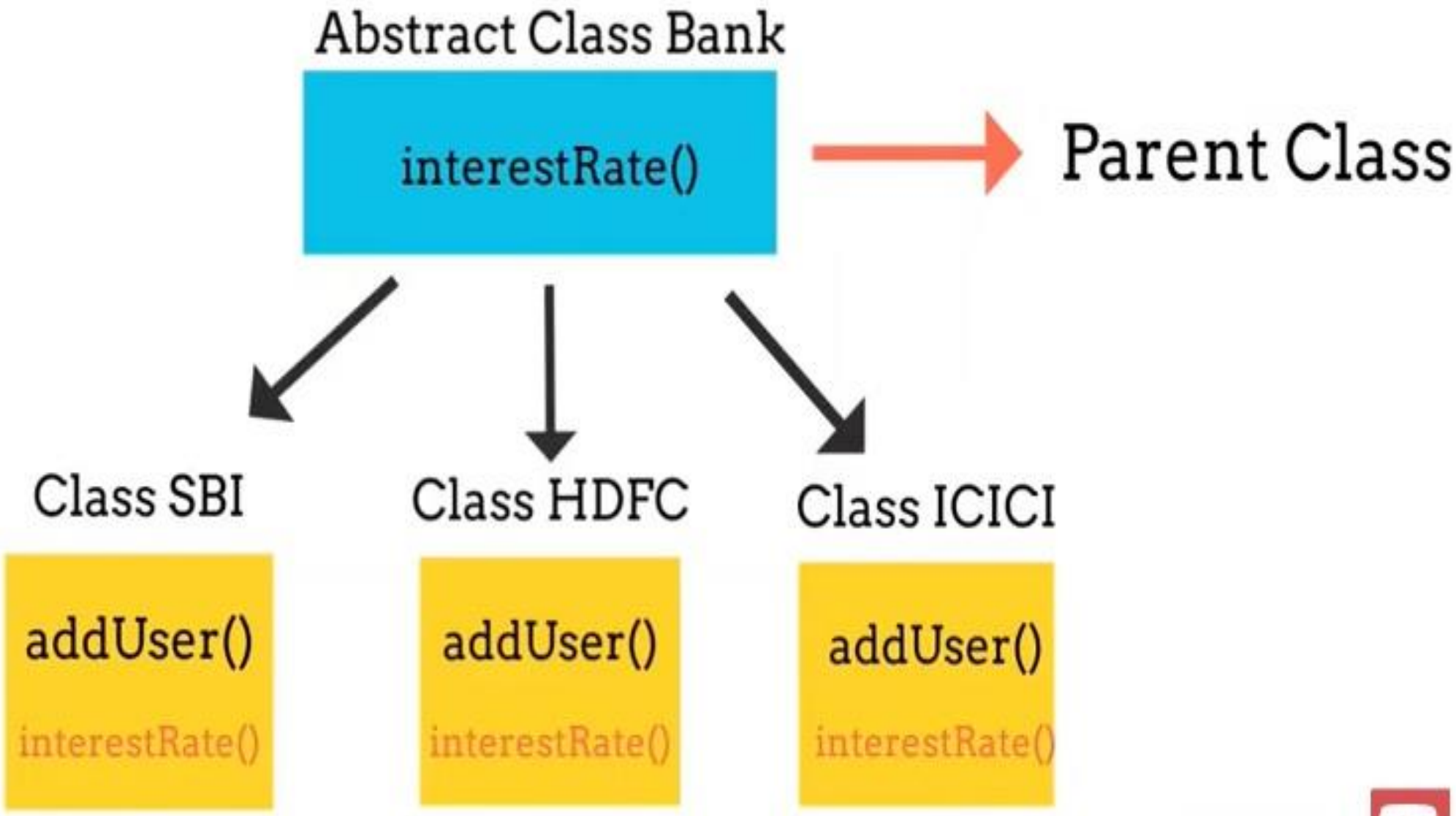
interestRate() = 10 %

Each class should have InterestRate() Method in it

Developer 1

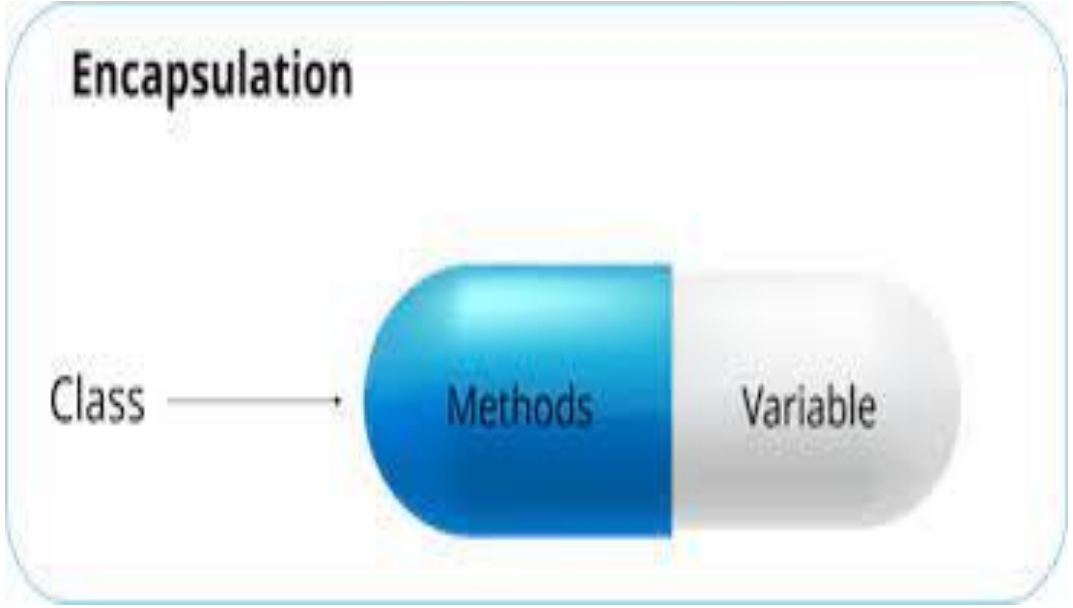
Developer 2

Developer 3


Abstract Class Bank
interestRate()
Parent Class
Class SBI
addUser()
interestRate()
Class HDFC
addUser()
interestRate()
Class ICICI
addUser()
interestRate()

Encapsulation

- Encapsulation is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.



Example of Encapsulation

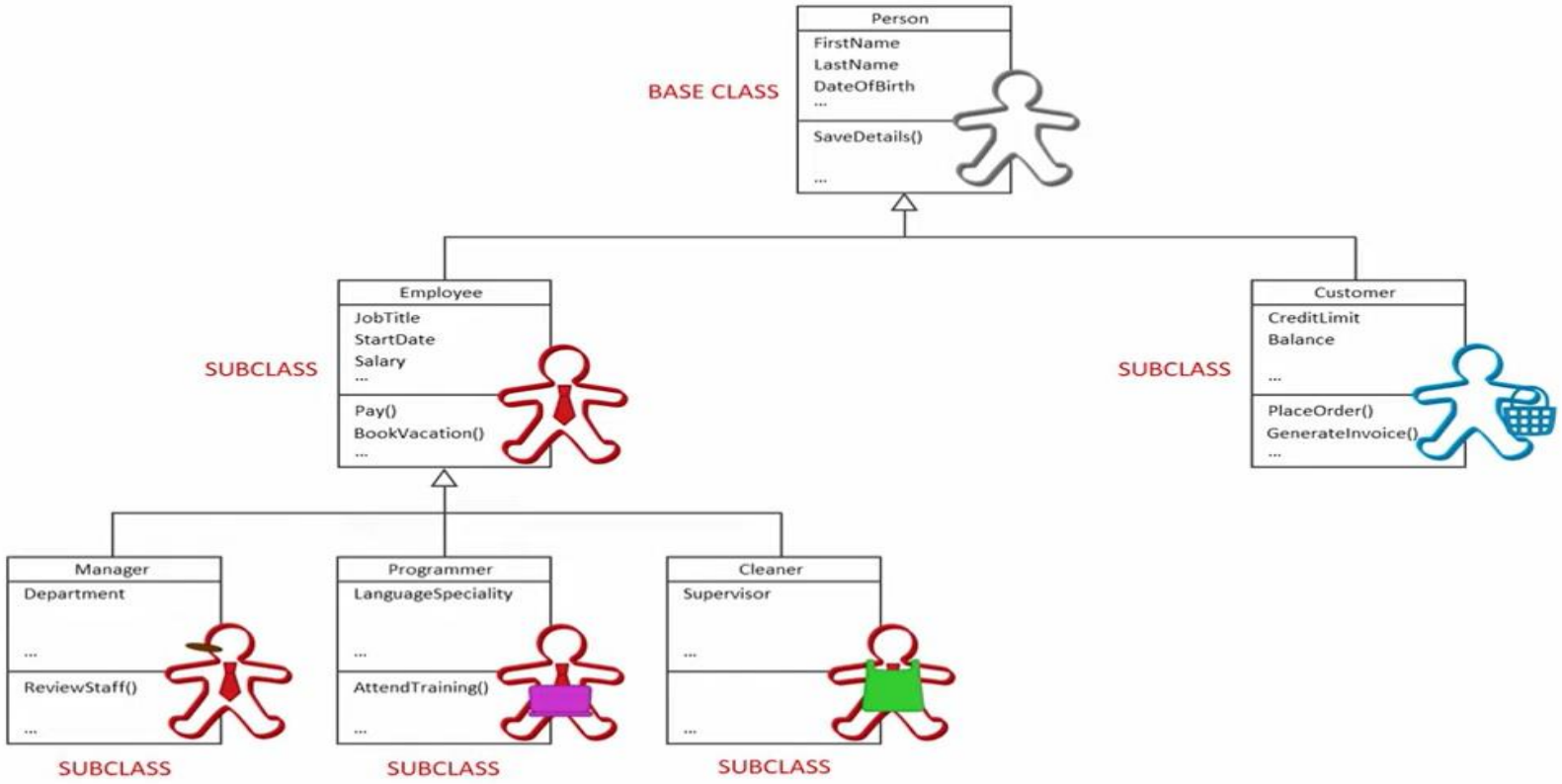
```
P6 = New Person

P6.SaveDetails()
P6.PaySalary()
```

Inheritance

- Inheritance is the process by which one object acquires the properties of another object.
- A deeply inherited subclass inherits all of the attributes from each of its ancestors in the class hierarchy.
- Top most class – Superclass
- Class inherited from superclass – Subclass

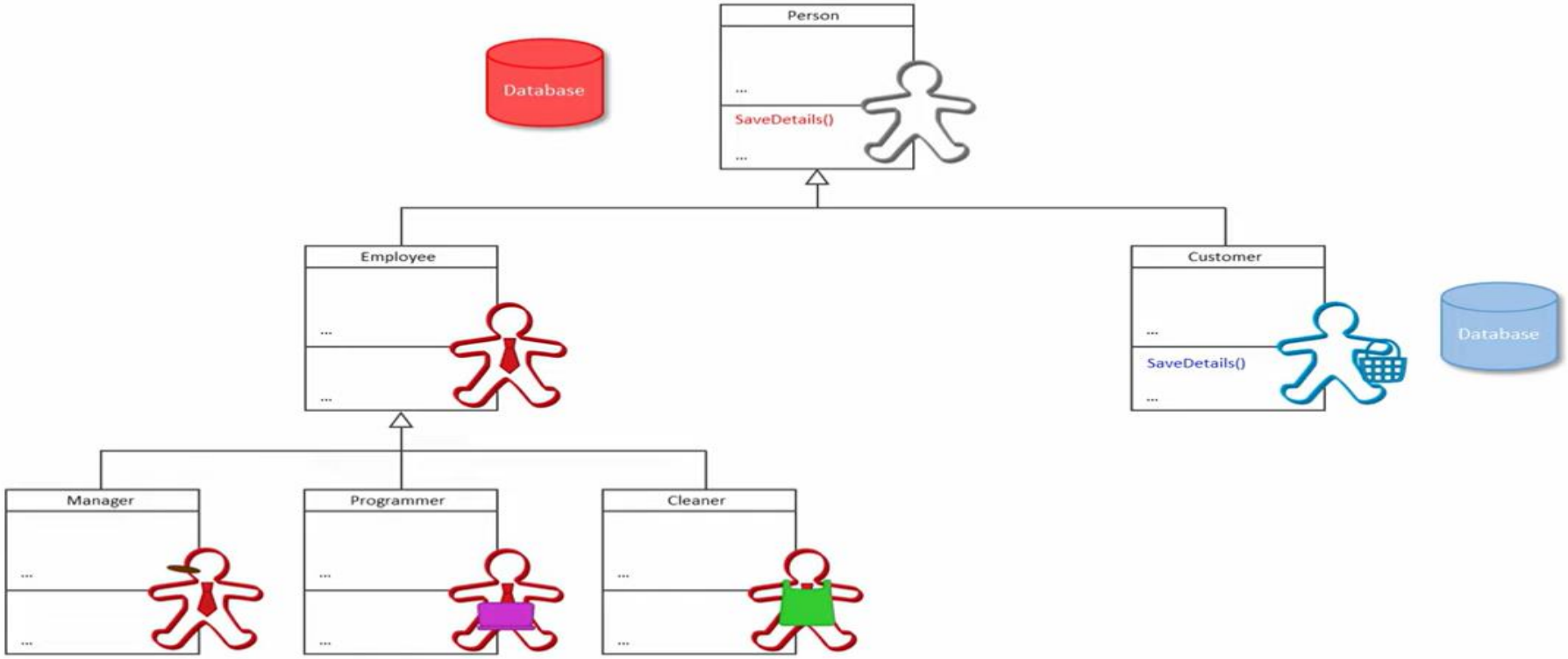
Inheritance



Polymorphism

- A class can implement an inherited method in its own way.
- “One interface, multiple methods.”
- For example, you can get an antilock braking system or traditional brakes, power or rack-and-pinion steering, and 4-, 6-, or 8-cylinder engines. Either way, you will still press the brake pedal to stop, turn the steering wheel to change direction, and press the accelerator when you want to move.

Polymorphism



OOP in Java

Abstraction

To simplify reality and focus only on the data and processes relevant to the application being built

Encapsulation

Data and the programs that manipulate those data are bound together and their complexity is hidden

Inheritance

A class can derive methods and properties from another class, resulting in a hierarchy of classes

Polymorphism

Different subclasses of the same superclass, can implement their shared interface in different ways

Characteristics of Java

Java is simple

- Java has done away with many complicated and lesser-used features like pointers, operator overloading, etc.
- Automatic garbage collection in java removes unused and memory-consuming object instances, without having to do that manually.

Object-Oriented

- Object-oriented programming (OOP) focuses on the objects that developers desire to handle, rather than the logic that is required to manipulate them.
- This kind of programming is ideally suited to big, complicated, and frequently updated or maintained projects.

Platform Independent

- Because it can operate on different systems, this byte code is platform-independent, i.e. Write Once, Run Anywhere (WORA).

Secure

- Java Program runs inside a virtual sandbox (Java Virtual Machine or JVM).
- **Classloader:** The Java Runtime Environment (JRE) includes a classloader that is used to dynamically load Java classes into the Java Virtual Machine. It improves security by isolating the package for local file system classes from those imported from network sources.

Robust

- The term “robust” means strong. But in a programmer’s dictionary, it means reliable.
- Certain types of error-prone programming constructs common in other languages have been removed in Java.
- It does not, for example, support pointers, which eliminates the risk of overwriting memory and damaging data.
- To provide programming support for resilience, Java has a runtime exception-handling mechanism.

Architectural Neutral

- Because there are no implementation-dependent aspects in Java, such as the size of primitive types, it is architecture-neutral.
- In C programming, the int data type takes up two bytes in 32-bit architecture and four bytes in 64-bit architecture.

Portable

- Java programs are portable because they are architecture-independent.
- They don’t need to be re-compiled to run on any platform. Java programs can be converted into byte code, which can be easily carried.

Performance

- Java’s performance and efficiency are remarkable since byte codes are very close to machine code.
- However, it is still slower than languages like C++ which are compiled.
- The fundamental reason is that Java applications, unlike C and C++ programs, run on a Java virtual machine (JVM) after compiling rather than directly on the computer’s CPU as native code.

Performance (Cont..)

- The performance of a Java bytecode compiled Java program is determined by how well the host Java virtual machine (JVM) manages its given tasks.
- And how well the JVM utilizes the computer hardware and operating system (OS) in doing so.

Distributed

- Distributed computing involves the collaboration of multiple computers over a network.
- Java was created with the goal of making distributed computing simple.
- Because networking is inherent in Java, developing network programs is as simple as sending and receiving data to and from a file.

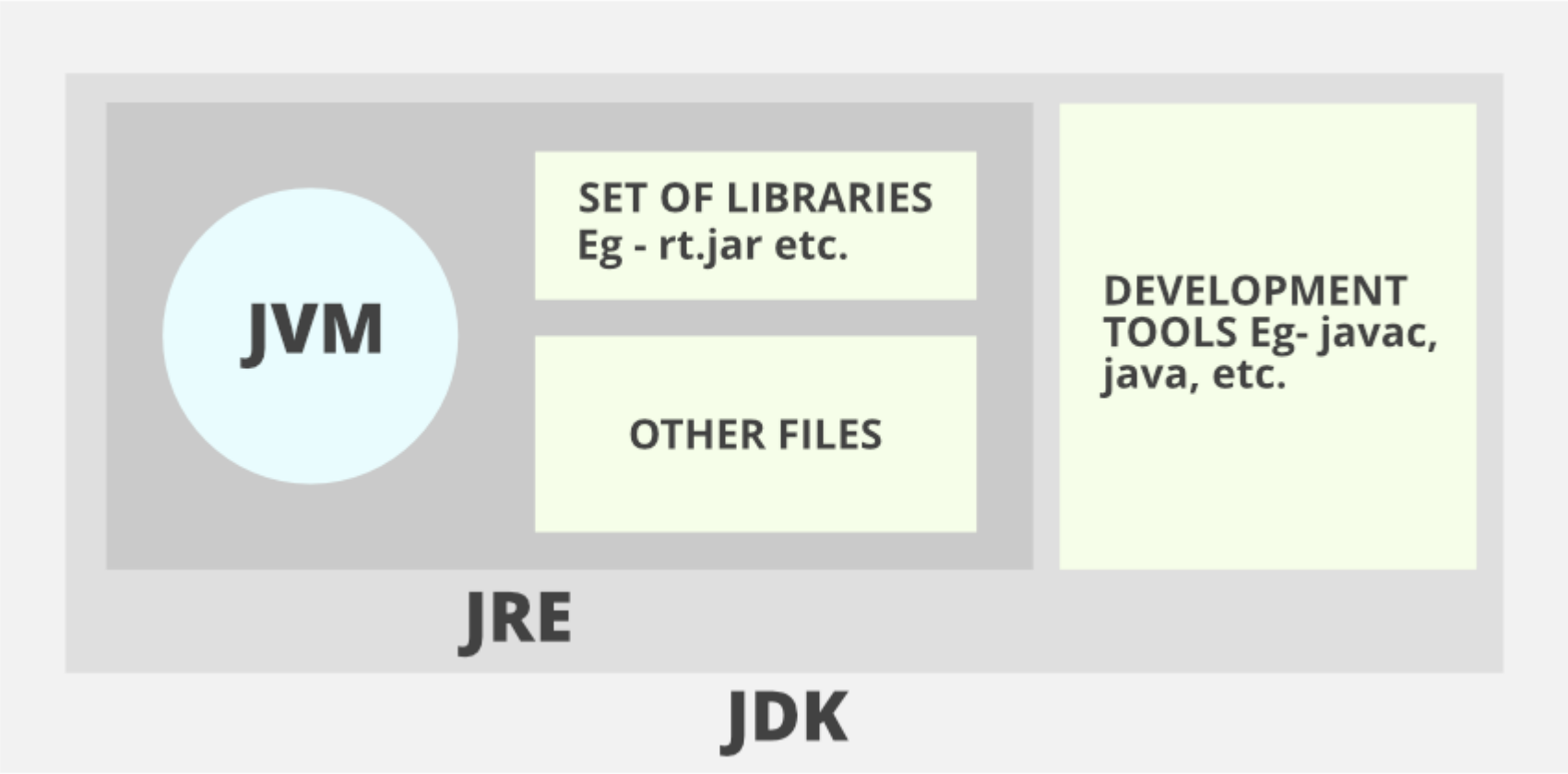
Multithreaded

- Multi-threading is a Java feature that facilitates the execution of two or more parts of a program at the same time to maximize CPU efficiency.
- These simultaneous processes can be implemented in Java through multi-threading.

The Java Environment

- Platform Independent - The meaning of platform-independent is that **the java compiled code(byte code) can run on all operating systems**.
- Whenever, a program is written in JAVA, the javac compiles it.
- The result of the JAVA compiler is the .class file or the bytecode and not the machine native code (unlike C compiler).
- The bytecode generated is a non-executable code and needs an interpreter to execute on a machine. This interpreter is the JVM and thus the Bytecode is executed by the JVM.
- And finally program runs to give the desired output.

The Java Environment



The Java Environment (Cont..)

- Java Run-time Environment (JRE) is the part of the Java Development Kit (JDK).
- It is a freely available software distribution which has Java Class Library, specific tools, and a stand-alone JVM.
- It is the most common environment available on devices to run java programs.

Java Source File Structure

eclipse-workspace1 - sample/src/sample/SourceFile.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

hi.java | India.java | helloworld.java | SourceFile.java | multipleString.java

```java
package sample;

public class SourceFile {

    public static void main(String args[])
    {
        System.out.println(args[0]);
    }
}
```

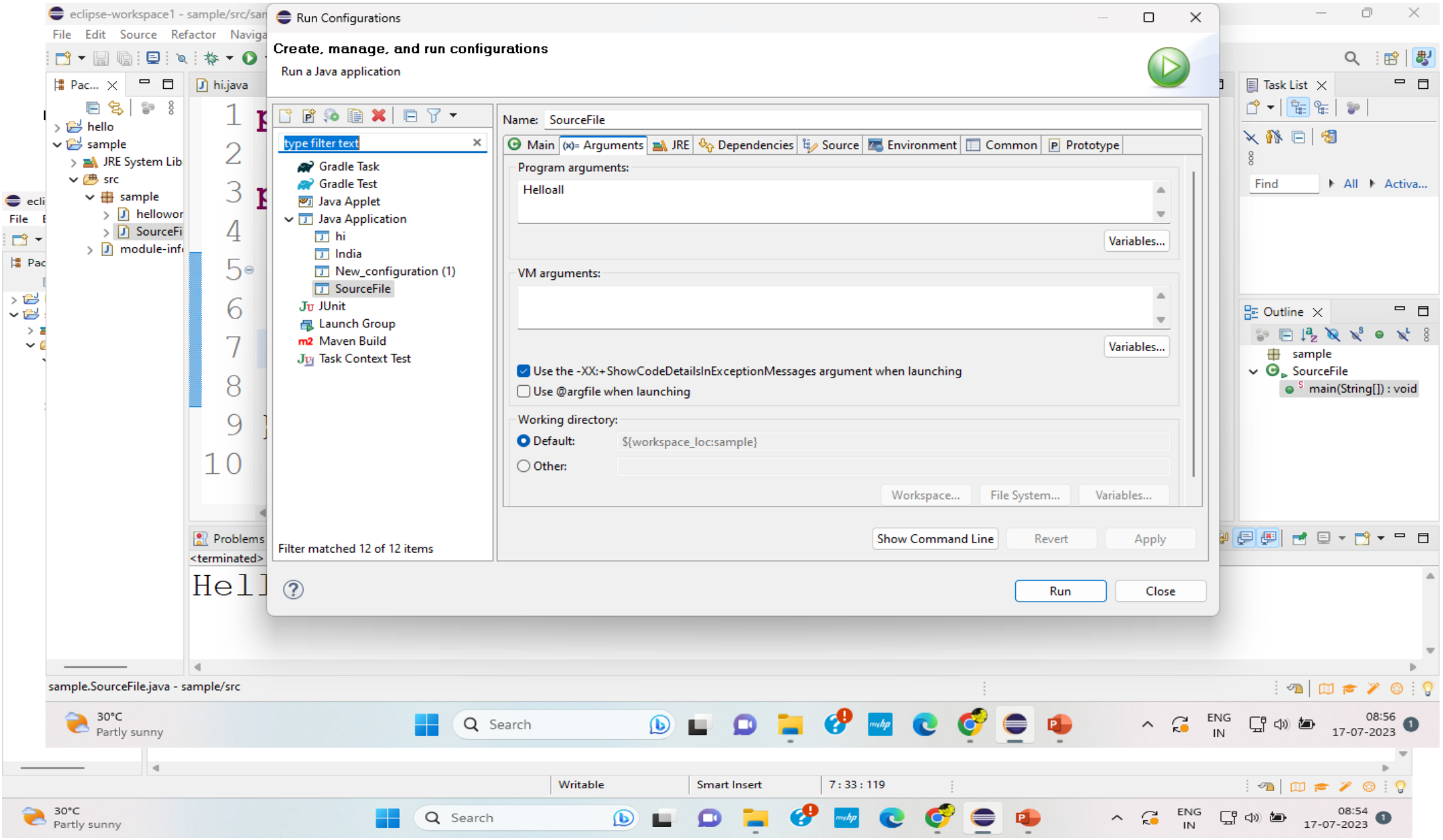
Problems | Javadoc | Declaration | Console

<terminated> SourceFile [Java Application] C:\Program Files\Java\jdk-18.0.1.1\bin\javaw.exe (17-Jul-2023, 9:03:24 am – 9:03:24 am) [pid: 15924]

```
Provided String Arguments:
hi
```

Writable | Smart Insert | 7 : 35 : 121

30°C Partly sunny | Search | ENG IN | 09:14 17-07-2023


eclipse-workspace1 - sample/src/sam
File Edit Source Refactor Naviga
Run Configurations
Create, manage, and run configurations
Run a Java application
type filter text
Gradle Task
Gradle Test
Java Applet
Java Application
hi
India
New_configuration (1)
SourceFile
JUnit
Launch Group
Maven Build
Task Context Test
Filter matched 12 of 12 items
Name: SourceFile
Main
Arguments
JRE
Dependencies
Source
Environment
Common
Prototype
Program arguments:
Helloall
Variables...
VM arguments:
Variables...
Use the -XX:+ShowCodeDetailsInExceptionMessages argument when launching
Use @argfile when launching
Working directory:
Default: ${workspace_loc:sample}
Other:
Workspace...
File System...
Variables...
Show Command Line
Revert
Apply
Run
Close
Pac...
hello
sample
JRE System Lib
src
sample
hellowor
SourceFi
module-inf
hi.java
Problems
<terminated>
Hell
Task List
Find
All
Activa...
Outline
sample
SourceFile
main(String[]) : void
sample.SourceFile.java - sample/src
Writable
Smart Insert
7 : 33 : 119
30°C
Partly sunny
Search
ENG
IN
08:56
17-07-2023
08:54

Compilation

```
D:\Java Example Programs>javac hello.java

D:\Java Example Programs>java hello
Hello, World!
```

Fundamental Programming Structures in Java

Java defines eight primitive datatypes

- byte
- short
- int
- long
- char
- float
- double
- boolean

Datatypes in Java

Data Type	Size	Description
`byte`	1 byte	Stores whole numbers from -128 to 127
`short`	2 bytes	Stores whole numbers from -32,768 to 32,767
`int`	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
`long`	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
`float`	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
`double`	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
`boolean`	1 bit	Stores true or false values
`char`	2 bytes	Stores a single character/letter or ASCII values

Defining classes in Java

```
<access specifier> class class_name
{
// member variables
// class methods
}
```

<access specifier> can be public, private or protected

Methods

- A method is a block of code that performs a specific task.
- Dividing a complex problem into smaller chunks makes your program easy to understand and reusable.
- In Java, there are two types of methods:
  - **User-defined Methods**: We can create our own method based on our requirements.
  - **Standard Library Methods**: These are built-in methods in Java that are available to use.

Declaring a Java Method

```
<returnType> methodName() {
// method body
}
```

- **<returnType>** - It specifies what type of value a method returns For example if a method has an int return type then it returns an integer value.
- **methodName** - It is an identifier that is used to refer to the particular method in a program.
- **method body** -It includes the programming statements that are used to perform some tasks. The method body is enclosed inside the curly braces { }. that are used to perform some tasks.

Complex syntax to declare a method

<modifier> static <returnType> nameOfMethod (parameter1, parameter2, ...) {

```
 //method body
}
```

<modifier> - It defines access types whether the method is public, private, or protected.

static - If we use the static keyword, it can be accessed without creating object.e.g.) Math.sqrt() can be directly called without creating object to Math class.

<returnType> - It specifies what type of value a method returns

parameter1/parameter2 - These are values passed to a method. We can pass any number of arguments to a method.

Standard Library Methods

- The standard library methods are built-in methods in Java that are readily available for use. These standard libraries come along with the Java Class Library (JCL) in a Java archive (*.jar) file with JVM and JRE.
- print() is a method of java.io.PrintSteam. The print("...") method prints the string inside quotation marks.
- sqrt() is a method of Math class. It returns the square root of a number.

Advantages of Methods

- The main advantage is **code reusability**. We can write a method once,and use it multiple times. We do not have to rewrite the entire code each time. Think of it as, "write once, reuse multiple times".
- Methods make code more **readable and easier** to debug.

Constructors

- A constructor in Java is similar to a method that is invoked when an object of the class is created. Although a constructor is called by the runtime implicitly when creating an object, it can also be called explicitly.
- At the time of calling constructor, memory for the object is allocated in the memory.
- It is a special type of method which is used to initialize the object.
- Every time an object is created using the new() keyword, at least one constructor is called.

Constructor Types

- **Default Constructor** - a constructor that is automatically created by the Java compiler if it is not explicitly defined.
- **No-Arg Constructor** - a constructor that does not accept any arguments
- **Parameterized constructor** - a constructor that accepts arguments

Default Constructor

- It calls a default constructor if there is no constructor available in the class. In such case, Java compiler provides a default constructor by default.
- The default constructor initializes any uninitialized instance variables with default values.
- For an example int will be initialized to 0

Default Values

Type	Default Value
`boolean`	false
`byte`	0
`short`	0
`int`	0
`long`	0L
`char`	\u0000
`float`	0.0f
`double`	0.0d
`object`	Reference null

Java No-Arg Constructors

- Similar to methods, a Java constructor may or may not have any parameters (arguments).
- If a constructor does not accept any parameters, it is known as a no-argument constructor. For example,

Parameterized Constructor

- A Java constructor can also accept one or more parameters. Such constructors are known as parameterized constructors (constructor with parameters).
- Here, we are passing values to the constructor. Based on the argument passed, the language variable is initialized inside the constructor.

Access Modifiers in Java

There are four types of access modifiers available in Java:

1. Default – No keyword required
2. Private
3. Protected
4. Public

Access Specifiers

1. **Private**: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
2. **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Access Modifiers Visibility

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

Java Static variable

- A static variable is associated with a class rather than an instance.
- A static field is declared using the static keyword.
- Static variables are shared across all instances of a class.
- There is only one copy of a static variable per class. Non-static variables cannot be called inside static methods.
- If any instance of a class modifies the value of a static variable, the change is reflected across all instances of the class. Static variable in java are memory-efficient as they are not duplicated for each instance..

Advantages of static variable

- It makes your program memory efficient (i.e., it saves memory).

Static Members

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Static block

- Whenever we use a static keyword and associate it to a block then that block is referred to as a static block.
- Java supports a special block, called a static block that can be used for static initialization of a class.
- This code inside the static block is executed only once: the first time the class is loaded into memory.

Comments

- Comments are used to make the program more readable by adding the details of the code.
- It makes easy to maintain the code and to find the errors easily.
- The comments can be used to provide information or explanation about the variable, method, class, or any statement.
- It can also be used to prevent the execution of program code while testing the alternative code.

Single-line Comments

- Single-line comments start with two forward slashes (//).
- Any text between // and the end of the line is ignored by Java (will not be executed).

```
// This is a comment
System.out.println("Hello World");
```

Multi-line Comments

- Multi-line comments start with /* and ends with */.

```
/* The code below will print the words Hello World
to the screen, and it is amazing *
****/
System.out.println("Hello World");
```

Javadoc Comments

- Javadoc is a tool which comes with JDK and it is used for generating Java code documentation in HTML format from Java source code, which requires documentation in a predefined format.
- Document tags that begin with an “at” sign (@) are called stand-alone tags (also called block tags), and they must be used on their own line.
- Tags that begin with a brace, such as {@code}, are called in-line tags, and they can be used within a larger description.

```
/** * The HelloWorld program implements an application that
 * simply displays "Hello World!" to the standard output.
 * * @author Zara Ali *
 @version 1.0 * @since 2014-03-31 */
public class HelloWorld
 {
 public static void main(String[] args)
 { // Prints Hello, World! on standard output.
 System.out.println("Hello World!");
}
}
```

Javadoc Tags

Tag	Meaning
@author	Identifies the author.
{@code}	Displays information as-is, without processing HTML styles, in code font.
@deprecated	Specifies that a program element is deprecated.
{@docRoot}	Specifies the path to the root directory of the current documentation.
@exception	Identifies an exception thrown by a method or constructor.
{@inheritDoc}	Inherits a comment from the immediate superclass.
{@link}	Inserts an in-line link to another topic.
{@linkplain}	Inserts an in-line link to another topic, but the link is displayed in a plain-text font.
{@literal}	Displays information as-is, without processing HTML styles.
@param	Documents a parameter.
@return	Documents a method's return value.
@see	Specifies a link to another topic.
@serial	Documents a default serializable field.
@serialData	Documents the data written by the **writeObject( )** or **writeExternal( )** methods.
@serialField	Documents an **ObjectStreamField** component.
@since	States the release when a specific change was introduced.
@throws	Same as **@exception**.
{@value}	Displays the value of a constant, which must be a **static** field.
@version	Specifies the version of a class.

Variables

- In Java, Variables are the data containers that save the data values during Java program execution.
- Every Variable in Java is assigned a data type that designates the type and quantity of value it can hold.
- A variable is a memory location name for the data.

Variables (cont..)

1.datatype: Type of data that can be stored in this variable.

2.data_name: Name was given to the variable.

- In this way, a name can only be given to a memory location. It can be assigned values in two ways:
- Variable Initialization e.g.) int i= 1;

- Assigning value by taking input ie.) getting input from the user.

Java Operators

- Arithmetic Operators
- Auto-increment & decrement Operators
- Bitwise Operators
- Logical Operators
- Assignment Operators
- Comparison or Relational Operators
- Ternary Operator
- Shift Operators

www.educba.com

Arithmetic Operators in Java

Operator	Result
+	Addition (also unary plus)
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
–=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
––	Decrement

Comparison or Relational Operators

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Logical Operators

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

Unary Operators

Operator Name	Symbol	Description	Example	Equivalent Expression
Unary Plus	+	It is used to represent the **positive** value.	+a	a
Unary Minus	-	It is used to represent the **negative** value.	-a	-
Increment Operator	++	It **increments** the value of a variable by **1**.	++a or a++	a=a+1
Decrement Operator	--	It **decrements** the value of a variable by **1**.	--a or a--	a=a-1
Logical Complement Operator	!	It **inverts** the value of a boolean variable.	!true	

Ternary Operator

- variable = (condition) ? expression1 : expression2
- If the condition is true then expression1 is evaluated else expression2 is evaluated.

Shift Operator in Java

Name of operator	Sign	Description
Signed Left Shift	<<	The left shift operator moves all bits by a given number of bits to the left.
Signed Right Shift	>>	The right shift operator moves all bits by a given number of bits to the right.
Unsigned Right Shift	>>>	It is the same as the signed right shift, but the vacant leftmost position is filled with 0 instead of the sign bit.

Assignment Operator in Java

- These operators are used to assign values to a variable.
- The left side operand of the assignment operator is a variable, and the right side operand of the assignment operator is a value.
- The value on the right side must be of the same data type of the operand on the left side.

a =10;

a =+ 10;

Bitwise Operator

Bitwise operators are used to performing the manipulation of individual bits of a number. They can be used with any integral type (char, short, int, etc.).

```
int a = 5;
        int b = 7;

        // bitwise and
        // 0101 & 0111=0101 = 5
        System.out.println("a&b = " + (a &
b));
```

Control Flow Structures in Java

1. Decision Making statements
    1. if statements
    2. switch statement
2. Loop statements
    1. do while loop
    2. while loop
    3. for loop
    4. for-each loop
3. Jump statements
    1. break statement
    2. continue statement

Arrays

- Array in java is a group of like-typed variables referred to by a common name.
- Arrays are stored in contiguous memory [consecutive memory locations].
- Since arrays are objects in Java, we can find their length using the object property length. This is different from C/C++, where we find length using sizeof.

```
int Array[] =new int [40];

int [] a
```

Syntax for Array Declaration

- There is a second form that may be used to declare an array: type[ ] var-name;
- Here, the square brackets follow the type specifier, and not the name of the array variable.
- For example, the following two declarations are equivalent:

```
int al[] = new int[3];
int[] a2 = new int[3];
```

- The following declarations are also equivalent:

```
char twod1[][] = new char[3][4];
char[][] twod2 = new char[3][4];
```

Packages

- Packages are containers for classes.
- They are used to keep the class name space compartmentalized.
- Package is both a naming and a visibility control mechanism.
- To create a package is quite easy: simply include a package command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package.
- The package statement defines a name space in which classes are stored. If you omit the package statement, the class names are put into the default package, which has no name.

Packages (Cont..)

- This is the general form of the package statement:

`package pkg;` Here, pkg is the name of the package.

```
import pkg1.pkg2.classname;
```

- Here, `pkg1` is the name of a top-level package, and `pkg2` is the name of a subordinate package inside the outer package separated by a dot (.).
- There is no practical limit on the depth of a package hierarchy, except that imposed by the file system.

An explicit classname or a star (*)indicates that the Java compiler should import the entire package.

```
import java.lang.*;
```

Packages (Cont..)

- Packages that come with JDK or JRD you download are known as built-in packages.
- The built-in packages have come in the form of JAR files and when we unzip the JAR files we can easily see the packages in JAR files, for example, lang, io, util, SQL, etc.
- Java provides various built-in packages for example java.awt, java.lang